A Survey: Efficient Decoding Algorithms for Non-Binary LDPC Codes

Akobir Kamolov, *KAIST*

***Abstract*—While binary Low-Density Parity-Check (LDPC) codes are prevalent in current telecommunications applications, Non-Binary (NB) LDPC codes offer superior performance, particularly at low Signal-to-Noise Ratio (SNR) values and with moderate code lengths. However, decoding algorithms for NB-LDPC codes often face challenges due to their high complexity. This paper investigates modern, efficient, and low-complexity decoding algorithms for NB-LDPC codes, providing both qualitative and quantitative comparisons of these methods.**

***Index Terms*—Non-Binary LDPC, decoding algorithm, low complexity.**

I. Introduction

COMMUNICATION systems consist of several complex components, and error-correcting codes (ECC) are a critical part of these components. ECC is used to encode a message before transmission through a channel and to decode the received message, which may be noisy due to transmission errors. The primary purpose of ECC is to minimize the impact of errors in data over the transmission channel by utilizing information redundancy, thereby eliminating the need for retransmission. A simple representation of an ECC system is illustrated in Fig. 1.

Error correction coding mechanisms are generally classified into three broad categories based on how they handle error detection, correction, and retransmission: Backward Error Correction (BEC), Forward Error Correction (FEC), and Hybrid Automatic Repeat Request (ARQ) [1]. FECs can be further divided into two main types: Convolutional Codes and Block Codes. Notable examples of block codes include Repetition Codes, Hamming Codes, Reed-Solomon Codes, and Low-Density Parity-Check (LDPC) Codes [2].

LDPC codes, introduced by Gallager in the 1960s, are among the few coding techniques that approach Shannon's limit for communication efficiency [3]. Although they were largely overlooked for many years due to their extremely high complexity, they were rediscovered by Neal and MacKay when modern computational power became sufficient for their implementation [4].

While binary LDPC codes are prevalent in applications such as 5G, satellite communications, and storage systems [5], Non-Binary LDPC (NB-LDPC) codes can offer superior performance under specific conditions, such as low signal-to-noise ratios (SNRs) and moderate code lengths. However, this improved performance comes at the cost of increased complexity and higher memory requirements [4].

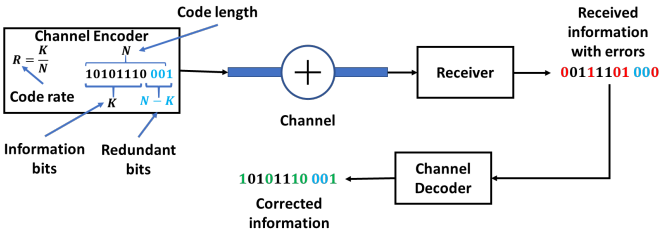


Fig. 1: A generic representation of an ECC System

The primary goal of this paper is to explore modern decoding algorithms for NB-LDPC codes. These algorithms aim to balance error-correction performance, computational complexity, and throughput, making them suitable for practical implementations.

II. Theoretical Background

A. Binary LDPC Codes

An LDPC code is a type of linear block code distinguished by its sparse parity-check matrix (PCM) $\mathbf{H}$, which can also be represented as a Tanner graph [6]. These codes encode K information bits into a codeword of length N bits. To enhance noise resistance and improve robustness, $M = N-K$ redundant bits are added, where $K < N$ [2]. The primary structural components of LDPC codes consist of check nodes (CNs) and variable nodes (VNs), which play a crucial role in the encoding and decoding processes. The number of check nodes and variable nodes can be determined from the rows and columns of the PCM. The non-zero elements in a PCM represent the connections (edges) between VNs and CNs. The Tanner graph of a PCM $\mathbf{H}$ with $M = 4$ and $N = 8$, along with the information exchange between nodes, can be illustrated as shown in Fig. 2.

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} \tag{1}$$

The code rate of $\mathbf{H}$ is defined as the ratio K/N, which is 0.5. An LDPC code is categorized as regular if there is an equal number of non-zero elements in all rows (d_c) and an equal number of non-zero elements in all columns (d_v). If d_c and d_v vary across rows and columns, the code is considered irregular [2]. The parity-check matrix (PCM) in (1) exemplifies a regular LDPC code. Regular codes typically demonstrate superior coding performance but are more complex computationally [7]. The sender uses LDPC codes to encode the message $\mathbf{v}$, of

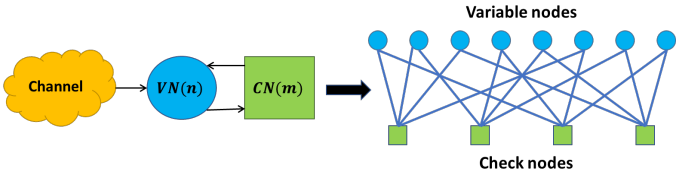
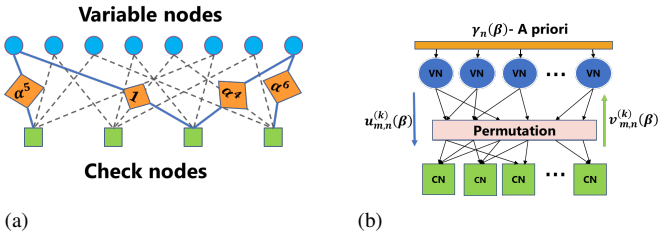


Fig. 2: Tanner representation of a **H**(4, 8) PCM in (1)



(a) (b)

Fig. 3: (a) Tanner representation of a **H**(4, 8) PCM in (4), (b) Illustration of SPA decoding algorithm

size K, by performing a binary multiplication with a generator matrix **G**, which results in a codeword **c**:

$$\mathbf{c} = \mathbf{vG} \tag{2}$$

On the receiver's side, the received message is multiplied by the parity-check matrix:

$$\mathbf{cH}^T = 0 \tag{3}$$

However, due to the errors caused by a noisy channel, the parity check equation (3) does not always hold. Therefore, binary LDPC decoding algorithms are employed to correct the errors in the received message.

B. Non-Binary LDPC Codes

The binary LDPC codes are called binary because the message elements can be represented by only two different values $\{0, 1\}$, and the arithmetic is also performed on a binary field. A binary field can be represented as $GF(2)$, which is a special case of Galois fields $GF(q)$, where q is the cardinality of the field. When $q = 2^p$, where p is a prime number, the Galois field $GF(q = 2^p)$ is said to be finite. Finite Galois fields hold certain key properties that make their elements remain within the field under Galois field operations, and the degree of Finite Galois field elements should always be less than the degree of the field itself [8].

A Galois field can be constructed using its irreducible primitive polynomials, whose roots generate all nonzero elements of the field under multiplication. Since the result of a multiplication of Finite Galois field elements is always another field element, multiplication is equivalent to permutation. In the Galois field $GF(2^p)$, the elements of the field can be represented by a polynomial $i(x) = \sum_{l=1}^{p} i_l x^{l-1}$ of degree $p-1$ with binary coefficients. Any set of p binary values $\{i_l\}_{l=1,\dots,p}$ determines a unique field element $i(x)$ [9]. For example, the Galois field of 4, $GF(2^2)$, has an irreducible primitive polynomial x^2+x+1, which generates field elements $\{0, 1, x, x+1\}$. The addition and multiplication tables over $GF(2^2)$ is shown Fig. 3.

Application of $GF(q)(q > 2)$ results in the representation of the messages by Galois field symbols rather than bits. In binary LDPC codes, a message could either be 0 or 1. But in NB-LDPC codes over $GF(2^p)$ a message can take up on any value among $\{0, 1, \alpha, \alpha^2, ..., \alpha^{p-2}\}$, where each symbol is represented by p bits [8]. Over $GF(2^3)$, the PCM in (1) might be represented as follows with a Tanner graph as shown in Fig. 4(a).

$$\mathbf{H} = \begin{bmatrix} \alpha^5 & \alpha^3 & \alpha^4 & 0 & 0 & \alpha^6 & 0 & 0 \\ 0 & \alpha^2 & 0 & \alpha & \alpha^6 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & \alpha^3 & 0 & \alpha^4 \\ 0 & 0 & \alpha^3 & 0 & \alpha^5 & 0 & \alpha^2 & \alpha^6 \end{bmatrix} \tag{4}$$

III. NB-LDPC Decoding Algorithms

This section explores the most popular NB-LDPC decoding algorithms existing in the literature. The original decoding algorithm of NB-LDPC codes, proposed by Davey and MacKay, is the sum-product algorithm (SPA), also known as the belief propagation algorithm (BP) [4]. SPA has three mathematically accurate variations that are implemented without any approximations: Fast Fourier Transform SPA (FFT-SPA) [10], Log Domain SPA [11], and mixed domain SPA [12]. Other popular approximated decoding algorithms are Extended Min-Sum algorithm [13], Min-Max algorithm [14], Trellis-based Min-Sum & Min-Max algorithms [15] [16]. The rest of this section is dedicated to the detailed study of the mentioned algorithms and their evaluations.

Before processing further, introducing certain definitions and mathematical representations of the messages are necessary:

- $u^{(k)}_{m,n}(\beta)-$ message vector from VN n to CN m during the kth iteration, where $\beta \in GF(q)$;
- $v^{(k)}_{m,n}(\beta)-$ message vector from CN m to VN n during the kth iteration, where $\beta \in GF(q)$;
- $S_c(n)-$ the set of CNs connected to the VN n;
- $S_v(m)-$ the set of VNs connected to the CN m;
- γ_n- *a posteriori* information from the channel;
- $K-$ maximum number of iterations;
- $h_{i,j}-$ the element of the PCM at the ith row and jth column;
- $L(m|a_n = \beta)-$ the (configuration) set of sequences of finite field elements a_j $(j \in S_v(m)\backslash n)$ such that $\sum_{j \in S_v(m)\backslash n} h_{m,j} a_j = h_{m,n}\beta$;
- $z^{(k)}_n-$ tentative decision of the nth received symbol during the kth iteration;

A. Sum-Product Algorithm

The SPA algorithm is also widely used in binary LDPC codes. In binary LDPC codes, the transmitted symbol can be either 0 or 1, meaning only a single reliability value needs to be processed. However, in $GF(q)$, the received symbol can

Algorithm 1 Sum-Product Algorithm [18]

Input: γ_n, $\mathbf{H}$, K
Initialize: $u^{(0)}_{m,n}(\beta) = \gamma_n(\beta)$, $z^{(0)}_n = \arg\max_{\beta}(\gamma_n(\beta))$
for $k = 1 : K$ **do**
  Compute $z^{(k-1)}\mathbf{H}^T$, **Stop if** $z^{(k-1)}\mathbf{H}^T = 0$
{*Check node processing*}
  for each check node m, each $n \in S_v(m)$, each $\beta \in GF(q)$ **do**

$$v^{(k)}_{m,n}(\beta) = \sum_{(a_j)\in L(m|a_n=\beta)} \left(\prod_{j\in S_v(m)\backslash n} u^{(k-1)}_{m,j}(a_j) \right) \quad (5)$$

  end for
{*Variable node processing*}
  for each variable node n, each $m \in S_c(n)$, each $\beta \in GF(q)$ **do**

$$u^{(k)}_{m,n}(\beta) = \gamma_n(\beta) \prod_{i\in S_c(n)\backslash m} v^{(k)}_{i,n}(\beta) \quad (6)$$

  end for
{*A posteriori information computation & tentative decision*}
  for each variable node n, each $\beta \in GF(q)$ **do**

$$z^{(k)}_n = \arg\max_{\beta} \gamma_n(\beta) \prod_{i\in S_c(n)} v^{(k)}_{i,n}(\beta) \quad (7)$$

  end for
end for

be any element of the Galois field, which requires the decoder to store and process $\log_2 q$ symbols and q reliability values. This, consequently, increases the complexity and the required memory of the decoder. Galois field additions and multiplication also require more complex computations, bringing the overall complexity to $O(d_c q^{d_c})$ [10].
The decoding process starts with updating the variable nodes with the received *aposteriori* information. Updated VN messages will go through the permutation network (multiplication in $GF(q)$). They will eventually be processed at the CN through the parity check equation and the tentative decision will be made. The CN messages are sent back through the permutation network, where they are de-permuted (division in $GF(q)$) before reaching the variable nodes [17].

B. FFT-based Sum-Product Algorithm

The complexity of the SPA increases significantly with higher values q. Since the messages are permuted at the permutation network and summed at the check node as shown in (5), this process is simply a convolution. Convolution can easily be calculated in the frequency domain as multiplication, reducing the complexity of the decoding process. But to satisfy this condition, a message should be an element of $GF(2^p)$. Then each message vector can be stored in a tensor of order 2 and dimension p : $U_{i_1,\ldots,i_p}$, where $i(x) \in GF(2^p)$ [9]. An example of tensorial representation and permutation of a message in $GF(8)$ is shown in Fig 4. Each dimension stores

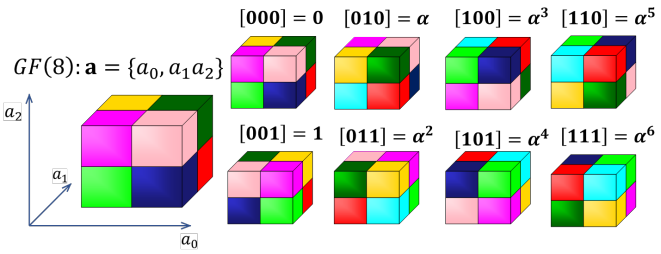


Fig. 4: Tensorial representation of a message over $GF(2^3)$

the reliability of $i(x)$ being 0 and 1. A permutation of the original message can be obtained by swapping the position of the non-zero dimension reliabilities in $\mathbf{a} = \{a_0, a_1, a_2\}$. For example, [010] can be obtained by swapping the orders across a_1 dimension [19]. The Walsh-Hadamard transform:

$$F(U) = U \times_1 F \times_2 F \ldots \times_p F, \quad F = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (8)$$

and a lemma for $GF(2^p)$ introduced in [10]

$$F(U * V) = F(U)F(V) \quad (9)$$

allow the check node to be processed completely in the frequency domain. As a result, the equation (5) becomes

$$v^{(k)}_{m,n}(\beta) = FFT^{-1}\left(\dot{\prod_{j\in S_v(m)\backslash n}} FFT(u^{(k-1)}_{m,j}(a_j)) \right) \quad (10)$$

where $\dot{\prod}$ represents term-by-term multiplication [17]. Converting convolution into multiplication through Fourier transform reduces the complexity to $O(d_c q \log_2 q)$. However, this complexity becomes overly complicated with higher orders of the Galois field ($q > 2$) [10].

C. Extended Min-Sum Algorithm

Extended Min-Sum algorithm extends the Binary Min-Sum algorithm into the non-binary domain. This algorithm uses log-likelihood ratios (LLRs) as a measure of reliability rather than probabilities. This method reduces multiplications to simple additions, contributing to the simplification of the overall complexity. For the nth received symbol, its LLR associated with $\beta \in GF(q)$ is defined as

$$\ln \frac{P(c_n = \hat{\beta}_n)}{P(c_n = \beta_n)} \quad (11)$$

where $\hat{\beta}_n$ is the $GF(q)$ element with the highest probability for the nth symbol. This makes the reliability of the most likely element always zero in the message vector, and the remaining reliabilities are always positive. Also, the elements with smaller LLR values are more likely to be the right symbol.
Avoiding complex operations (multiplications, exponentials, logarithms, etc.) does not always result in the lowest complexity. To further simplify the algorithm, it is also useful to compute the output reliability using only a limited number of the largest incoming values. To achieve this, from the message vector of length q, the highest $n_r << q$ reliabilities can be

Algorithm 2 Extended Min-Sum Algorithm [18]

Input: γ_n, $\mathbf{H}$, K
Initialize: $u_{m,n}^{(0)}(\beta) = \gamma_n(\beta)$, $z_n^{(0)} = \arg\min_{\beta}(\gamma_n(\beta))$
for $k = 1 : K$ **do**
Compute $z^{(k-1)}\mathbf{H}^T$, **Stop if** $z^{(k-1)}\mathbf{H}^T = 0$
{*Check node processing*}
for each check node m, each $n \in S_v(m)$, each $\beta \in GF(q)$ **do**

$$v_{m,n}^{(k)}(\beta) = \min_{(a_j)\in L(m|a_n=\beta)} \left(\sum_{j\in S_v(m)\backslash n} u_{m,j}^{(k-1)}(a_j) \right) \quad (12)$$

end for
{*Variable node processing*}
for each variable node n, each $m \in S_c(n)$, each $\beta \in GF(q)$ **do**

$$u_{m,n}^{'(k)}(\beta) = \gamma_n(\beta) + \sum_{i\in S_c(n)\backslash m} v_{i,n}^{(k)}(\beta) \quad (13)$$

$$u_{m,n}^{(k)}(\beta) = u_{m,n}^{'(k)}(\beta) - \min_{\omega\in GF(q)}(u_{m,n}^{'(k)}(\omega)) \quad (14)$$

end for
{*A posteriori information computation & tentative decision*}
for each variable node n, each $\beta \in GF(q)$ **do**

$$z_n^{(k)} = \arg\min_{\beta}(\gamma_n(\beta) + \sum_{i\in S_c(n)} v_{i,n}^{(k)}(\beta)) \quad (15)$$

end for
end for

Algorithm 3 Trellis Extended Min-Sum Algorithm [18]

Input: γ_n, $\mathbf{H}$, K
Initialize: $u_{m,n}^{(0)}(\beta) = \gamma_n(\beta)$, $z_n^{(0)} = \arg\min_{\beta}(\gamma_n(\beta))$
for $k = 1 : K$ **do**
Compute $z^{(k-1)}\mathbf{H}^T$, **Stop if** $z^{(k-1)}\mathbf{H}^T = 0$
{*Delta domain conversion*}
for each check node m, each $n \in S_v(m)$, each $\beta \in GF(q)$ **do**

$$\Delta u_{m,n}^{(k-1)}(\eta_n = \beta^{'(k-1)} + \beta) = u_{m,n}^{(k-1)}(\beta^{'(k-1)}) - u_{m,n}^{(k-1)}(\beta) \quad (16)$$

end for
{*Syndrome computation*}
for each check node m, each $n \in S_v(m)$ **do**

$$\omega_m^{(k-1)} = \sum_{i\in S_v(m)} \Delta u_{m,i}^{(k-1)}(\beta^{'(k-1)})) \quad (17)$$

end for
{*Check node processing*}
for each check node m, each $n \in S_v(m)$, each $\beta \in GF(q)$ **do**

$$\Delta U^{(k-1)}(\beta) = \min_{\eta(\beta)\in conf(n_r,n_c)} \left(\sum_{i\in S_v(m)\backslash n} \Delta u_{m,i}^{(k-1)}(\eta_i(\beta)) \right) \quad (18)$$

$$\Delta v_{m,n}^{(k)}(\beta + \eta_n(\beta)) = \min_{\eta(\beta)\in conf(n_r,n_c)} \{\Delta v_{m,n}^{(k)}(\beta + \eta_n(\beta), \Delta U^{(k-1)}(\beta) - \Delta u_{m,n}^{(k-1)}(\eta_n(\beta))\} \quad (19)$$

$$v_{m,n}^{(k)}(\beta + \omega_m^{(k-1)} + \beta_n^{'(k-1)}) = \Delta v_{m,n}^{(k)}(\beta) \quad (20)$$

end for
end for

chosen to process, while the rest are discarded. In case of high d_c or n_r, the highest $n_c \leq d_c - 1$ reliable check nodes are selected, while the rest are discarded. This significantly reduces the complexity to $O(n_c n_r q)$ [9].
However, the EMS algorithm is not optimal, which makes it prone to performance degradation compared to the SPA algorithm. To avoid the EMS algorithm rapidly converging to a local minimum or a pseudo-codeword, the introduction of an offset to compensate for the overestimated reliabilites is necessary.

D. Trellis Extended Min-Sum Algorithm

Selecting the n_r and n_c and creating the right configuration set is a complex process. Trellis EMS algorithm uses a systematic approach to make the most optimal configuration set $conf(n_r, n_c)$ for decoding. Such an optimal configuration set is generated by creating a 0-order path from the most reliable elements in each received message vector. Other less reliable paths are called deviations. The set $conf(n_r, n_c)$ means that the paths used to compute the syndrome consist of n_r most reliable message from a single message vector and n_c elements that deviate from the 0-order configuration path [16]. The values of n_r and n_c depend on the design parameters. The messages from the check nodes to the variable nodes are processed using the difference in the reliability of the configurations ($\Delta u_{m,n}$) and the syndrome ($\Delta U_{m,n}$) [17]. These simplifications further reduce the overall complexity to $O(\binom{q-1}{n_c} n_r^{n_c})$ [15]. Since the complexity is independent of d_c, the Trellis EMS algorithm results in nearly the same complexity and latency for all coding rates.
The EMS and Trellis EMS algorithms can further be simplified to Min-Max and Trellis Min-Max algorithms to increase throughput. What is required is to replace the summation operations in (12) and (18) with a max comparison [18]:

$$\sum_{j\in S_v(m)\backslash n} \Rightarrow \max_{j\in S_v(m)\backslash n} \quad (21)$$

IV. Results & Analysis

Several methods have been employed in this paper to compare the decoding algorithms. To obtain the best configuration set of Trellis EMS for a PCM $\mathbf{H}$ (60, 40) over the $GF(64)$, Monte Carlo simulations have been performed with 100,000 frames [20]. According to the simulation results as shown in Fig. 5, $n_r = 20$ is the lowest optimum value, at which performance degradation is negligible.
A comparison of decoding algorithms has also been performed

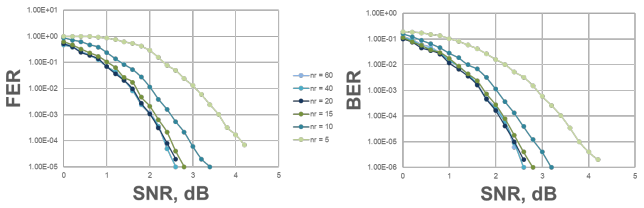


Fig. 5: FER and BER performance of Trellis EMS with varying n_r values

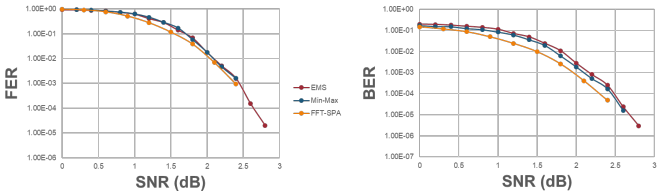


Fig. 6: Simulated comparison of EMS, Min-Max, and FFT-SPA algorithms

with EMS, Min-Max, and FFT-SPA algorithms. The Monte Carlo simulations with 100,000 frames have been carried out on a PCM $\mathbf{H}$ (64, 32) over the $GF(256)$ [20]. The simulation results indicate that the differences in error correction performance among the selected algorithms are nearly negligible, as shown in Fig. 6. Although FFT-SPA demonstrated slightly better performance than the other algorithms, the performance differences would become more apparent if a more suitable choice of PCM and Galois field order was utilized.

V. Conclusion & Future Research

This paper provides a detailed analysis of the most popular modern Non-Binary LDPC decoding algorithms, discussing their performance, complexity, implementations, and limitations. It also includes a quantitative comparison of selected decoding algorithms, offering deeper insights into their error-correction capabilities.

Although existing literature does provide comparisons of these decoding algorithms, there are underlying issues with such comparisons. The design parameters of decoders can vary significantly, and some studies fail to report these parameters accurately or use misleading figures, leading to less reliable results. Additionally, decoder architectures are often implemented in different environments, which complicates comparisons. Therefore, the absence of a standardized framework for simulating and comparing these algorithms makes it difficult to achieve reliable results. Future research could focus on developing a standard framework and establishing uniform design parameter choices for decoding architectures.

References

[1] R. W. Hamming, "Error detecting and error correcting codes," in *The Bell System Technical Journal*, vol. 29, no. 2, pp. 147-160, April 1950, doi: 10.1002/j.1538-7305.1950.tb00463.x.

[2] Todd K. Moon, "Error Correction Coding: Methematical Methods and Algorithms," 2005

[3] R. Gallager, "Low-density parity-check codes," in *IRE Transactions on Information Theory*, vol. 8, no. 1, pp. 21-28, January 1962, doi: 10.1109/TIT.1962.1057683.

[4] M. C. Davey and D. J. C. MacKay, "Low density parity check codes over GF(q)," 1998 *Information Theory Workshop* (Cat. No.98EX131), Killarney, Ireland, 1998, pp. 70-71, doi: 10.1109/ITW.1998.706440.

[5] F. Li et al., "Review on 5G NR LDPC Code: Recommendations for DTTB System," in *IEEE Access*, vol. 9, pp. 155413-155424, 2021, doi: 10.1109/ACCESS.2021.3121587.

[6] C. Poulliat, M. Fossorier, and D. Declercq, "Design of regular (2, d_c)-LDPC codes over GF(q) using their binary images," *IEEE Trans. Commun.*, vol. 56, no. 10, pp. 1626–1635, Oct. 2008

[7] M. G. Luby, M. Mitzenmacher, M. A. Shokrollahi, and D. A. Spielman,"Improved low-density parity-check codes using irregular graphs," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 585–598, Feb. 2001.

[8] G. L. Mullen and D. Panario, *Handbook of Finite Fields*. Boca Raton, FL, USA: CRC Press, 2013

[9] D. Declercq and M. Fossorier, "Decoding Algorithms for Nonbinary LDPC Codes Over GF(q)," in *IEEE Transactions on Communications*, vol. 55, no. 4, pp. 633-643, April 2007, doi: 10.1109/TCOMM.2007.894088.

[10] L. Barnault and D. Declercq, "Fast decoding algorithm for LDPC over $GF(2^q)$," *Proceedings 2003 IEEE Information Theory Workshop* (Cat. No.03EX674), Paris, France, 2003, pp. 70-73, doi: 10.1109/ITW.2003.1216697.

[11] H. Wymeersch, H. Steendam, and M. Moeneclaey, "Log-domain decoding of LDPC codes over GF(q)," in *Proc. IEEE Int. Conf. Commun.*, vol. 2, 2004, pp. 772–776

[12] C. Spagnol, E. M. Popovici, and W. P. Marnane, "Hardware implementation of $GF(2^m)$ LDPC decoders," *IEEE Trans. Circuits Syst.* I, Reg. Papers, vol. 56, no. 12, pp. 2609–2620, Dec. 2009.

[13] X. Zhang and F. Cai, "Reduced-complexity decoder architecture for non-binary LDPC codes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 19, no. 7, pp. 1229–1238, Jul. 2011.

[14] V. Savin, "Min-max decoding for non binary LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory*, 2008, pp. 960–964.

[15] E. Li, D. Declercq, and K. Gunnam, "Trellis-based extended minsum algorithm for non-binary LDPC codes and its hardware structure," *IEEE Trans. Commun.*, vol. 61, no. 7, pp. 2600–2611, Jul. 2013.

[16] J. O. Lacruz, F. García-Herrero, D. Declercq, and J. Valls, "Simplified trellis min-max decoder architecture for nonbinary low-density parity-check codes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1783–1792, Sep. 2015.

[17] X. Zhang, *VLSI Architectures for Modern Error-Correcting Codes*. Boca Raton, FL, USA: CRC Press, 2017.

[18] O. Ferraz et al., "A Survey on High-Throughput Non-Binary LDPC Decoders: ASIC, FPGA, and GPU Architectures," in *IEEE Communications Surveys Tutorials*, vol. 24, no. 1, pp. 524-556, Firstquarter 2022, doi: 10.1109/COMST.2021.3126127.

[19] Y. Toriyama, *High-Performance and Energy-Efficient Decoder Design for Non-Binary LDPC Codes*. Diss. UCLA, 2016.

[20] M. Helmling, S. Scholl, F. Gensheimer, T. Dietz, K. Kraft, S. Ruzika, and N. Wehn. *Database of Channel Codes and ML Simulation Results*. www.rptu.de/channel-codes, 2023.